

This member-only story is on us. [Upgrade](#) to access all of Medium.

Member-only story

Featured

My 5-Minute Workflow to Find Bugs on Any Website

A step-by-step guide to my most effective, shortcut methods for bug bounty hunting.



Lostsec

Follow

11 min read · Sep 27, 2025

1.5K

17



Introduction

Hi everyone, welcome back! Today, I'm going to show you the exact method I use to find bugs on almost any website in under five minutes. I'll show you exactly how I do it. I use a really fast shortcut that combines a few clever tricks to quickly understand a website and then I let automated tools do the hard work of scanning for bugs. It's all about working smart, not hard, so you can find the most important vulnerabilities without wasting any time.

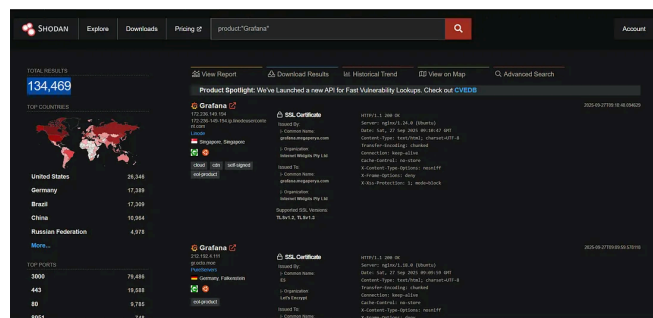
In this walkthrough, I'll cover:

- How I use **Shodan** to quickly identify mass-scale CVE exposures.
- Scripts that uncover hidden inputs, forms and URLs.
- Automation workflows with **Nuclei**, **GF patterns**, **Uro** and other tools.
- Recon techniques with **WaybakURLs**, **AlienVault**, **URLScan**, **VirusTotal** and more.
- My own custom scripts like **Lost Uncover** and **LostFuzzer** to streamline scanning.

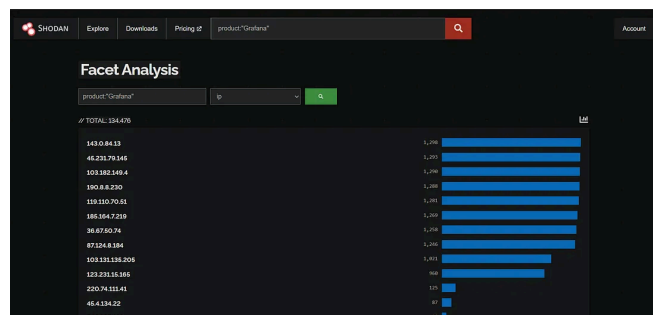
Method 1: Mass Scanning with Shodan & Nuclei

This is my go-to method for finding recently disclosed CVEs at a massive scale. It's incredibly efficient for identifying low-hanging fruit across thousands of targets.

1. **Find Your Target CVE:** First, pick a CVE you want to hunt for. For this example, let's say we're looking for a specific vulnerability in a popular software.
2. **Shodan Dorking:** Head over to **Shodan** and use a specific search dork related to the product or CVE. Shodan will instantly show you all the internet-connected devices matching your query.



3. **Facet Analysis for IPs:** On the results page, click the “More” option to open the Facet Analysis tab. From there, select the “ip” option. This neatly organizes all the results by their IP address.



4. **Extract and Scan:** Now for the magic. I use a custom bookmarklet I wrote that automatically fetches all the IP addresses from the Shodan results and downloads them as a .txt file. Here is the that bookmarklet script.

for ip's:

```
javascript:(function(){var ipElements=document.querySelectorAll('strong');var ip
```

Medium

Search

Get app

Write

Notifications

Profile

Home

Library

Profile

Stories

Stats

Following

Find writers and publications to follow.
[See suggestions](#)

```
javascript:(function(){var ipElements=document.querySelectorAll('strong'),ips=[]
```

You can also use **shef**, a lightweight command-line tool written in Go by our team. Shef integrates Facets into your terminal environment and operates without requiring an API key.

GitHub — thehaq/shef: bring shodan facets into your terminal without API key.

bring shodan facets into your terminal without API key. — thehaq/shef

github.com

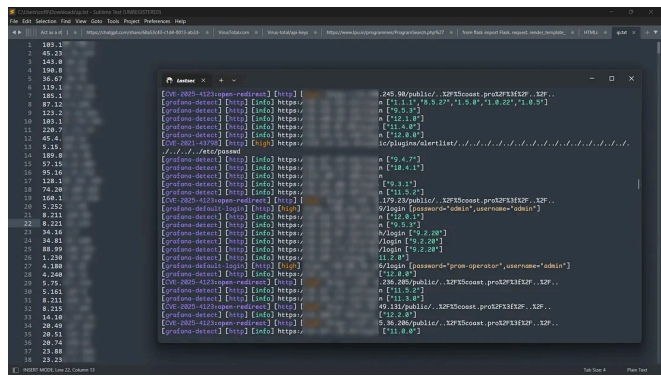
q/shef

gets into your terminal without API

0 Issues 65 Stars 21 Forks

Once you have the file, you can feed it directly into **Nuclei** for automated scanning. Simply replace the tags or template name with the one relevant to your CVE. In minutes, Nuclei will scan the entire list and highlight any vulnerable hosts.

```
cat ip.txt | nuclei -tags grafana -bs 50 -c 50 -es info
```



Method 2: Uncovering What's Hidden in Plain Sight

Developers often disable form inputs, buttons or hide entire sections of a page, thinking they are secure. This script helps you bypass those client-side restrictions.

```
javascript:(function(){document.querySelectorAll('[disabled],[readonly]').forEach(function(e){e.removeAttribute('disabled');e.removeAttribute('readonly');});})
```

I've integrated a handy script into my **Lost Uncover** tool that automatically finds and modifies these elements on a webpage:

```
javascript:(function(){if(document.getElementById('lostsec-scanner'))return;let els=document.querySelectorAll('[disabled],[readonly]');els.forEach(function(e){e.removeAttribute('disabled');e.removeAttribute('readonly');});})
```

You can use this HTML file to test the script. Simply open the file and use the 'Lost Uncover → Unhide Element' option

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Bookmarklet Test Page</title>
  <style>
    body {
      background-color: black;
      color: #00ff88;
      font-family: monospace;
      padding: 20px;
    }
    input, button {
      font-family: monospace;
      margin-top: 5px;
      margin-bottom: 20px;
      padding: 5px;
    }
    .hidden {
      display: none;
    }
    .grayed {
      pointer-events: none;
      opacity: 0.4;
    }
  </style>
</head>
<body>
  <h1>Bookmarklet Test Page</h1>

  <h2>Disabled Input</h2>
  <label>Email (Disabled):<br>
  <input type="text" value="you@nowhere.com123" disabled>
</label>

  <h2>Readonly Input</h2>
  <label>Username (Readonly):<br>
  <input type="text" value="readonly_user123" readonly>
</label>

  <h2>Hidden Button</h2>
  <button class="hidden" id="secret-btn">Secret Admin Button</button>

  <h2>Grayed-Out Section</h2>
  <div class="grayed">Premium Content</div>
</body>
</html>
```


This is where my main automation begins. The first step is to get a complete map of the target's web presence, and for that, I use a script that queries **AlienVault's Open Threat Exchange (OTX)**.

[illegible]

Its superpower is its thoroughness. It digs deep and fetches every known URL associated with a domain, crawling through pages until it has a massive list. Once I have this raw data, I refine it to find the most interesting targets:

- The final command looks something like this, leaving me with a clean list of potentially vulnerable URLs ready for testing:

You can find all the gf patterns in my GitHub repository. Make sure to install the gf tool first before using them

GitHub - tomnomnom/gf: A wrapper around grep, to help you

grep for things

A wrapper around grep, to help you grep for things -
tomnomnom/gf

github.com

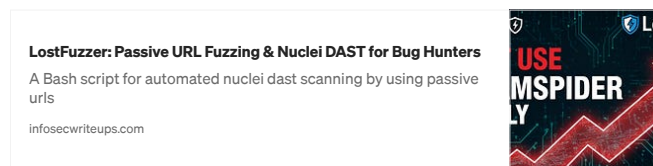
omnom/gf

nd grep, to help you grep for things

🔍 4 ⭐ 2k 📄 337
Used by Items Forks

You can use my **LostFuzzer** script for a simple, direct approach. Provide a domain (or a list of domains) and it will automatically run a Nuclei DAST scan to find vulnerabilities. It's lightweight, easy to use, and delivers high-

impact results with minimal effort. You Read more about this in my Medium article:

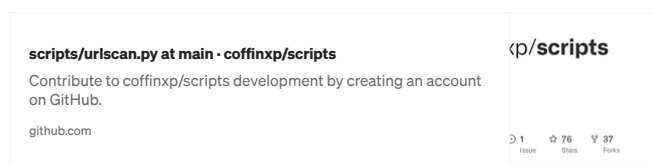
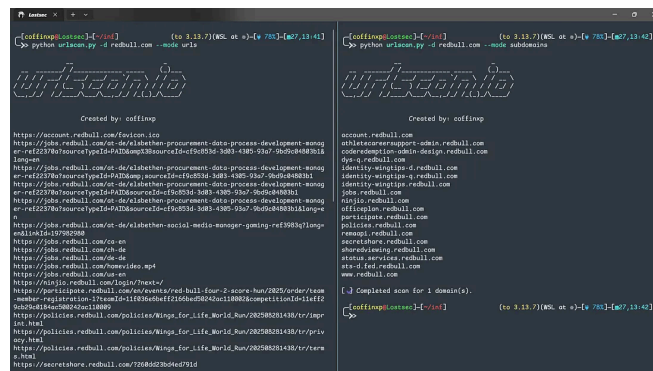


URLScan.io: Uncovering Hidden Subdomains and Endpoints

URLScan.io is another goldmine for reconnaissance, and using a script to automate queries is a huge time-saver. This tool is great for two main tasks:

- **Finding Subdomains:** You can run the script in “Subdomains” mode to get a list of all related subdomains. I often pipe this output directly to **HTTPX** to see which ones are live and what technology they’re running.
- **Discovering More URLs:** In “URLs” mode, it fetches another unique set of URLs. You can add these to the list you got from AlienVault or scan them separately with Nuclei DAST.

```
python urlscan.py -d redbull.com --mode urls
python urlscan.py -d redbull.com --mode subdomains
```



VirusTotal Script: Mining for Digital Gold

This is one of my secret weapons for finding the kind of sensitive information that other tools often miss. Using a script written by Orwa that queries **VirusTotal**, you can uncover some incredible findings.

Because VirusTotal analyzes files and URLs submitted by users worldwide, its database sometimes contains exposed secrets related to your target. I have personally used this script to find:

- Email and password combinations.
- Internal API keys.
- Password reset tokens and other sensitive links

It’s always worth a quick check, you never know what secrets might be hiding in plain sight, also make sure to add your three different virustotal api keys in the given script.

```
./virustotal.sh domain.com
```


Gospider: fast crawling & JS harvesting

Here's another fast crawler I use for site mapping, JS analysis and endpoint discovery. Lightweight and perfect for one-liners with jsleaks, lazyegg, gf patterns and nuclei, it can generate and verify links from JavaScript using LinkFinder, detect AWS S3 buckets from page sources and extract subdomains or hidden URLs. It also pulls data from the Wayback Machine, Common Crawl, VirusTotal and AlienVault. giving a complete view of your target's exposed assets for a fast, powerful recon workflow.

Basic commands:

```
gospider -s https://example.com # Crawl the site and print
gospider -s https://example.com -a # Crawl the site and also g
(Archive.org,CommonCrawl,V
gospider -s https://example.com -d 3 # Crawl the site with recur
gospider -s https://example.com --subs # Crawl the site and includ
gospider -s https://example.com --sitemap -d 2 # Try to parse sitemap.xml
gospider -s https://example.com -p http://127.0.0.1:8080 # Crawl the site but s
gospider -s https://example.com/upload/ -d 3 --whitelist "/upload/" # Crawl onl
```

```
[coffinxp@lostsec]~$
$ gospider -s https://www.cutout.pro
[robots] - https://www.cutout.pro/css/
[robots] - https://www.cutout.pro/js/
[robots] - https://www.cutout.pro/remove-background/upload?fileId=*
[robots] - https://www.cutout.pro/*remove-background/upload?fileId=*
[robots] - https://www.cutout.pro/ai-video-generation/upload?originUrl=*
[robots] - https://www.cutout.pro/*ai-video-generation/upload?originUrl=*
[url] - [code-200] - https://www.cutout.pro
[subdomains] - http://www.cutout.pro
[subdomains] - https://www.cutout.pro
[aws-s3] - cutout.s3.amazonaws.com
[href] - https://www.cutout.pro/favicon.ico
[href] - https://fonts.googleapis.com/css?family=Roboto:100,300,400,500,700,900&display=swap
[href] - https://cdn.jsdelivr.net/npm/gmdi/font@latest/css/materialdesignicons.min.css
[href] - https://cdn.promeai.pro/cutout-web/v2.6.17.in/3cd0942.js
[href] - https://cdn.promeai.pro/cutout-web/v2.6.17.in/8c94fe1.js
```

Pro JS One-Liners for Hunting Secrets:

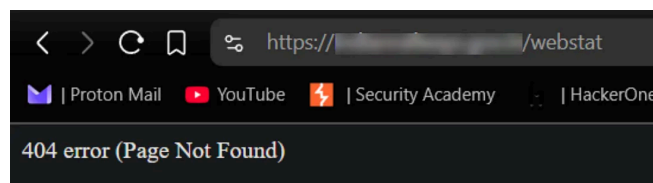
```
gospider -s https://example.com | grep -Eo 'https://[^\s\<>[:space:]]+' | sor
# Crawl domain, extract absolute URLs, then sort & dedupe.

gospider -s https://example.com -d 3 | grep '\.js$' | grep -Eo 'https://[^\s\<>[:space:]]+' | sor
# Crawl to depth 3, keep .js URLs, extract absolute URLs.

gospider -s https://example.com | grep -Eo 'https://[^\s\<>[:space:]]+' | gre
# Find JS file URLs and scan them with jsleaks for possible secrets (jsleaks fla

gospider -s https://example.com | grep -Eo 'https://[^\s\<>[:space:]]+' | gre
# For each JS URL, print a header and run lazyegg to extract endpoints, domains,

gospider -s https://example.com | grep -Eo 'https://[^\s\<>[:space:]]+' | gre
# Feed discovered JS URLs into Nuclei using the credentials-disclosure-all templ
```





Written by *Lostsec*

9.8K followers · 0 following

Follow



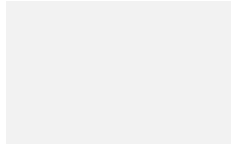
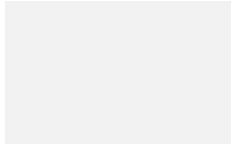
Helping organizations stay secure through Bug Hunting, OSINT and Security Research | Sharing knowledge as a Content Creator



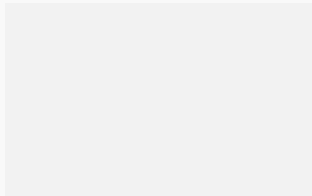
Placeholder text for bio or header

Placeholder text for bio or header

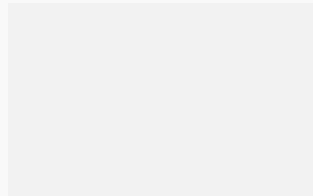
Placeholder text for bio or header



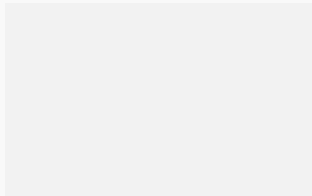
Placeholder text for bio or header



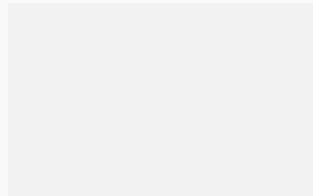
Placeholder text for bio or header



Placeholder text for bio or header



Placeholder text for bio or header



Placeholder text for bio or header